Lab Experiment – 2

Error Detection and Correction

Submission Date : 16.08.2024

1. Hamming code error detection and Correction
2. Cyclic Redundancy Check
3. Checksum

1. Implement the following Hamming Code Error detection and correction technique with an input of 110001 and options are no error, error in data and error in redundancy bits.

- Get Input
- Write code to find no. of r bits
- Print codeword
- Print Syndrome bits without error bits
- Print Syndrome bits with error.

Verify the following sample pattern

- Sender Source ----------
  - $r_0 = a_0+a_1+a_2$ modulo 2 arithmetic
  - $r_1 = a_1+a_2+a_3$
  - $r_2 = a1+a_3+a_0$
- Receiver - Destination
  - $s_0 = b_0+b_1+b_2 + q_0$ **modulo 2 arithmetic**
  - $s_1 = b_1+b_2+b_3 + q_1$
  - $s_2 = b1+b_3+b_0 + q_2$
- Error detection and Correction

Sample I/O

Input = 0001

Sender output code word = 0001101

Receiver input = 0001101

Output – No Error

Receiver Input – 0001100

Output – Error Detected

Corrected Output 0001101

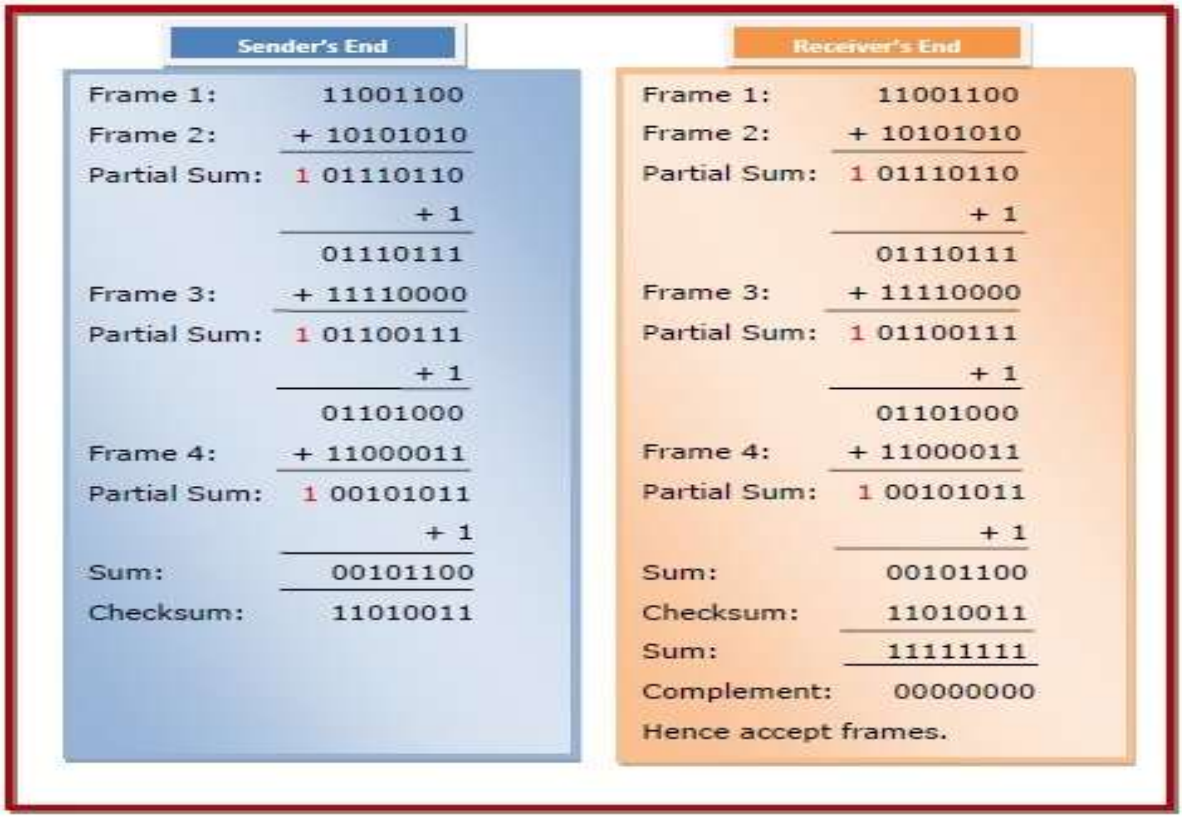
2. A cyclic redundancy check (CRC) is an error-detecting code commonly used in digital networks and storage devices to detect accidental changes to raw data. Sample I/O is given below :

```
11010011101100 000 <--- input right padded by 3 bits
1011               <--- divisor
01100011101100 000 <--- result
 1011              <--- divisor ...
00111011101100 000
  1011
00010111101100 000
   1011
00000001101100 000
       1011
00000000110100 000
        1011
00000000011000 000
         1011
00000000001110 000
          1011
00000000000101 000
           101 1
-----------------
00000000000000 100 <---remainder (3 bits)
11010011101100 100 <--- input with check value
1011               <--- divisor
01100011101100 100 <--- result
 1011              <--- divisor ...
00111011101100 100

00000000001110 100
          1011
00000000000101 100
           101 1
-----------------
                 0 <--- remainder
```

- Write a program to implement Cyclic Redundancy Check CRC using Binary Division or Polynomial Division.
- Introduce Error bit in Sender Data.
- Find Remainder at Receiver and print output as "Error in data" or not.

3. Implement check for the sample data given for Binary Checksum of 32 bit or Choose first 8 characters of your name for the Hex checksum:



Input : Binary input and Divisor

Output : Checksum at Sender and Receiver site with error detection